AKDENIZ UNIVERSITY · FACULTY OF ENGINEERING · DEPARTMENT OF COMPUTER ENGINEERING

CSE 491 Midterm Report

GiraffeGraph design and progress at midterm. The extraction and review prototype exists; independent annotation and expert assessment have not started. The remaining semester will establish a human reference and validate the prototype against the proposal targets.

Project title	**A Human-in-the-Loop Knowledge Graph Generation Engine for Preventing Hallucination and Schema Drift in LLMs: An Application on Hadith Isnad Chains**
Group no.	10
Project supervisor	Res. Asst. Dr. Taha Yiğit Alkan
Repository URL	https://github.com/GiraffeGraph/giraffegraph
Submission date	8 November 2026

Group members

#	Name and surname	Student no.	Role / responsibility
1	Yahya Efe Kuruçay		Architecture, checks, decision records and evaluation
2	Deniz Büyükşahin		Annotation guideline, annotator 1 and review interface
3	Alperen Ulukaya		Isnad segmenter, annotator 2 and chain view

1. Changes Since the Proposal

The CSE 491 topic, team, stack and core deliverables remain unchanged [1]. Two updates affect execution. The broader TÜBİTAK research plan is kept separate from this semester's acceptance criteria [2].

What changed	From → to	Reason
Annotation schedule	Weeks 6–8 → recovery in Weeks 9–10	Independent annotation and expert assessment have not started; their outputs cannot be reported as completed.
Research framing	Broad supporting studies → hadith validation as the main evaluation	The November research proposal focuses on checks, human review and narrator decision reuse. Supporting simulations remain preliminary evidence.
Midterm assessment	Report and presentation → report discussed at Supervisor Meeting 4	The current assignment requires no separate midterm presentation.

2. Problem and Objectives Short Recap

An LLM may propose narrator names that are absent from its evidence, alter a written name, or produce types outside the schema. Even a name present in the text may identify the wrong person. GiraffeGraph separates extraction proposals from accepted graph facts and makes the source and acceptance basis inspectable. Its target users are hadith researchers and students, with engineers as a secondary audience.

Project I targets 50 independently double-labelled Arabic hadiths, adjudication by domain experts, a 100-hadith end-to-end prototype and reproducible comparisons of extraction, checks and review [1]. It records transmission chains without judging religious authenticity. Authority-based identity resolution, larger multi-model experiments and graph-based question answering remain later work; the six-month research plan extends the same system [2].

3. Requirements

The following IDs retain the proposal's requirement numbering. "In development" means that prototype code exists but group acceptance is incomplete; it does not mean that a requirement has passed final verification. C1–C7 refer to the components in Section 4.2 [1], [3].

3.1 Functional requirements

ID	Requirement	Priority	Status
FR-1	The system shall ingest each hadith with a stable source identifier and content hash. C1.	Must	In development
FR-2	The system shall segment isnad chains and return narrator spans with source offsets. C2.	Must	In development
FR-3	The system shall request schema-constrained candidates separately for each detected chain. C2.	Must	In development
FR-4	The system shall reject an unknown candidate type individually and record a schema proposal without discarding valid siblings. C3.	Must	In development
FR-5	The system shall route extracted candidates to review when their quotation is absent from the source segment. C3.	Must	In development
FR-6	The system shall route a narrator candidate to review when its name is absent from its quotation after declared normalization. C3.	Must	In development
FR-7	The system shall flag a candidate that treats the Prophet as an ordinary narrator. C3.	Should	Planned
FR-8	The system shall display source evidence and check reasons and allow accept, reject and edit decisions. C4.	Must	In development
FR-9	The system shall record reviewer, action, timestamp and decision duration for every measured review decision. C4.	Must	In development
FR-10	The system shall commit accepted candidates and preserve separate transmission links for different hadiths and chain positions. C5.	Must	In development
FR-11	The system shall display a committed chain and the quotation and acceptance basis for each link. C6.	Should	In development
FR-12	The system shall report precision, recall and F1 against a fixed reference under strict and loose matching. C7.	Must	In development
FR-13	The system shall support versioned node, relation and property definitions and reject invalid database identifiers. C1.	Could	In development

3.2 Non functional requirements

ID	Requirement	How it will be verified
NFR-1	Every canonical fact shall have source evidence and a traceable human decision or named acceptance policy.	S2 audit of every fact; negative tests for manual entry and missing evidence. C3–C6.
NFR-2	Repeating a commit shall not duplicate nodes or keyed transmission links.	Real-database retry and failure tests, with before/after counts. C5.
NFR-3	After dependencies are installed, the in-memory backend suite shall run offline in less than one minute.	Timed pytest run in CI. Current local run: 3.98 seconds; real DB tests excluded.
NFR-4	A new group member shall start the prototype using only the repository README and example configuration.	Clean setup exercise in Week 14; record commands and failures.
NFR-5	The repository shall contain no API keys or deployment passwords.	Repository and history secret scan before release; local secrets remain in ignored environment files.

4. System Architecture and Data Model

The backend is a layered modular application with domain entities, application use cases and ports, infrastructure adapters, and HTTP/worker interfaces. This keeps extraction providers and stores replaceable while retaining one validation policy. Next.js communicates with the FastAPI API using JSON over HTTP. The worker executes queued extraction, validation and commit jobs through the same use cases [3].

4.1 Architecture diagram

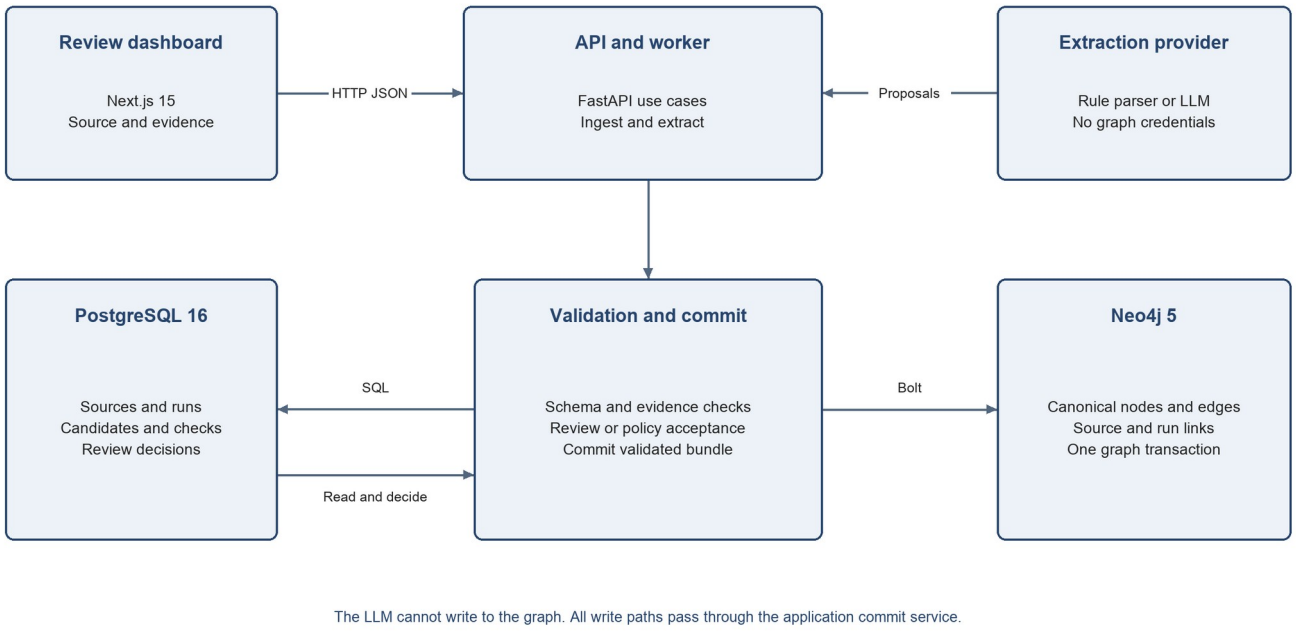


Figure 1. Current prototype architecture. The dashboard exchanges review actions through the API; PostgreSQL holds operational records and Neo4j holds committed graph facts. Arrows show application data paths, not direct browser database access.

4.2 Components

Component	Responsibility	Interfaces / depends on
C1 Project and ingestion	Version schemas; load sources; retain identifiers, content hashes and text.	Project/schema/source API; PostgreSQL repositories.
C2 Segmentation and extraction	Detect chain spans; call a provider; persist candidate nodes, edges and evidence.	SegmentationStrategy and LLMProvider ports; external model HTTP API.
C3 Candidate validation	Check types, endpoints, keys, properties, quotation containment and value containment.	Schema and source reads; ValidationResult and ReviewItem writes.
C4 Human review	Show evidence; append accept/reject/edit decisions; retain before/after edit values.	Review API; PostgreSQL decision records.
C5 Canonical commit	Refuse unresolved review; commit validated bundles and graph deltas.	CanonicalGraphRepository; Neo4j driver.
C6 Chain exploration	Read chains, neighboring narrators, source evidence and acceptance basis.	Read API; Neo4j plus operational records.
C7 Evaluation	Compare candidates and chains with reference files; aggregate metrics and bootstrap intervals.	Offline JSONL files, run exports and Python scorer.

PostgreSQL stores the job queue; no separate message broker is required for the preliminary prototype. The LLM receives only the relevant text and schema and has no graph credentials. In-memory repository adapters support offline development, while Docker Compose runs the persistent configuration. The publicly accessible landing page is separate from the controlled research application.

4.3 Data model

The operational store distinguishes raw evidence, machine proposals and human decisions. A Project owns schema versions and source units. A SourceUnit is linked to a schema version and contains the raw hadith, external reference and content hash. Its Segment records identify chain slices and source offsets. Each ProcessingRun identifies its source, schema and pipeline version, so a result can be interpreted against the configuration that produced it [3].

Entity or record	Key fields and relationships	Storage
Project and SchemaVersion	project_id; schema_version_id; node/relation/property definitions; merge keys; active/version metadata. One project has many versions.	PostgreSQL
SourceUnit and Segment	source_unit_id, external_ref, content_hash, text; segment_id, source_unit_id, start/end offsets. One source has many segments.	PostgreSQL
ProcessingRun and ProcessingJob	run_id, source_unit_id, schema_version_id, pipeline version and status; job type, run link and execution status.	PostgreSQL
CandidateNode and CandidateRelation	candidate_id, run_id, type, properties, confidence and status; relation endpoint references. EvidenceRef links to a segment, quotation and optional offsets.	PostgreSQL
ValidationResult and ReviewItem	run/candidate reference, rule/reason code and outcome; review item identifies the candidate or schema proposal requiring a decision.	PostgreSQL
ReviewDecision	review item and run references; accept/reject/edit, reviewer_id, timestamp, optional duration_ms and note; before/after values for edits. Append-only history.	PostgreSQL
Canonical node and edge	project/type identity; narrator name or hadith reference; keyed transmission identity; source/segment/run provenance.	Neo4j
GraphDelta	run reference and summary of graph changes; links support audit and commit reporting.	PostgreSQL and graph-side commit metadata

Graph schema and relation identity

The isnad template defines Narrator and Hadith nodes, NARRATED_FROM edges between narrators, and TRANSMITS edges from narrators to hadiths. Narrator uses name as its merge key and Hadith uses hadith_ref. A NARRATED_FROM edge includes hadith_ref, chain_no and position as merge keys; sigha stores the transmission wording. Thus two links between the same written names remain distinct when they belong to different chains or positions.

This is a surface-name graph. Exact name identity can merge different people with the same name or split one person across variant names. The prototype must not describe these nodes as resolved historical identities. An authority-linked narrator ledger and biographical consistency rules are later extensions, consistent with the broader research proposal [2].

Data lifecycle and consistency

The candidate lifecycle is pending → validated, review or rejected. A reviewer may accept, reject or edit a review item; revalidation preserves the latest recorded human decision. Commit writes a run's accepted bundle in one Neo4j transaction. PostgreSQL and Neo4j do not share a distributed transaction: a failure between graph commit and metadata update still needs reconciliation and retry verification. There is no committed-run rollback.

Human decisions currently target candidate IDs from a specific run. A fresh extraction does not automatically reuse them. Acceptance records for automatically validated candidates are derived on request rather than stored as immutable snapshots. Before expanding human review, we will stabilize decision targeting and preserve enough policy/version information to reconstruct each acceptance.

External data and services

Sanadset supplies Arabic hadith text for development [5]. Ifta-derived labels support preliminary comparison only; AR-Sanad is used in separate identity simulations. Each downloaded input has a manifest entry with its hash and origin. Source editions and redistribution rights require review before publication. Only required chain text is sent to configured LLM providers; cached responses and prompt/model identifiers support repeatable analysis.

5. Technology Selection

The selected stack remains the proposal stack [1]. The alternatives below explain the design tradeoffs; no comparative performance benchmark is claimed. Keeping the existing stack preserves working adapters and tests, while the application ports allow later replacements [3].

Layer / need	Chosen technology	Alternatives considered	Reason for the choice
Backend	Python 3.12, FastAPI, Pydantic	Django REST Framework; Flask	Python fits text processing and evaluation. Typed API validation and generated OpenAPI documentation reduce custom request handling [7].
Operational store	PostgreSQL 16, SQLAlchemy, Alembic	SQLite; storing all records in Neo4j	Runs, candidates and decisions have explicit relational ownership and migration needs. PostgreSQL supports transactional updates [8]; SQLite remains less representative of the shared deployment.
Canonical graph	Neo4j 5	PostgreSQL edge tables; RDF triplestore	Nodes and keyed edges fit narrator chains and path exploration [6], [9]. Existing hadith ontologies inform the representation. The cost is operating and reconciling two stores.
Dashboard	Next.js 15, React 19, TypeScript	Plain React/Vite; Streamlit	The existing routed dashboard separates upload, extraction, review and reading. Typed components can reuse the same API [10]; migration has no demonstrated benefit this term.
Extraction	Rule-based provider plus configurable LLM adapters	One fixed model SDK; fine-tuned extractor	The rule parser is an offline baseline. Ports isolate external APIs and let us compare prompts/models without changing commit logic. Fine-tuning requires a larger reference set.
Execution	PostgreSQL job queue and worker	Celery/Redis; synchronous-only pipeline	A persistent queue supports staged work with fewer services. The synchronous batch endpoint remains useful for small runs, but needs timeout checks.
Build and checks	uv lockfile, pytest, ruff, mypy; npm lockfile	Unpinned pip installs; manual-only checks	Locked dependencies and offline repository adapters support repeatable tests. Static checks catch different failures from runtime tests.
Local deployment	Docker Compose; GitHub Actions CI	Manual database installation; Kubernetes	Compose describes the five local services. Kubernetes would add deployment work beyond the current single-team prototype. CI checks backend and frontend changes.

Why deterministic checks and human review coexist

Schema membership answers whether a proposal fits the allowed representation, not whether it follows from a source. Quotation containment checks whether the quoted evidence exists. Value containment checks whether the narrator name is supported by that quotation. A failed check requests review rather than proving that the narrator is wrong; grammatical variation and inferred kinship names can also trigger it. Human-and-LLM graph validation has prior work [11]; this project measures the contribution of the individual checks on the same candidates.

The Arabic normalizer is currently shared by grounding checks. Its folding rules can hide spelling distinctions, so the raw source and offsets must remain available to the reviewer. The design therefore keeps original text, records reasons and reports strict and loose matching separately. Neither a high confidence score nor a passed containment check is sufficient evidence of the correct historical identity.

Scope of the preliminary prototype

The semester deliverable is an inspectable extraction-to-graph workflow. It does not include a public multi-user deployment, narrator authenticity judgement or a completed RAG question-answering interface. The broader research plan still requires 300 human-labelled hadiths, three models including an open-weight model, repeated runs and real decision-reuse measurements [2]; those are not claimed as achieved by this report.

6. User Interface Approach

The current Turkish dashboard presents four ordered steps within a project: Metin yükle, Zincirleri çıkar, İncele and Zincirleri oku. Project creation can apply the isnad schema. Advanced tools expose schema editing, jobs and the raw graph without making them prerequisites for the main review workflow [3].

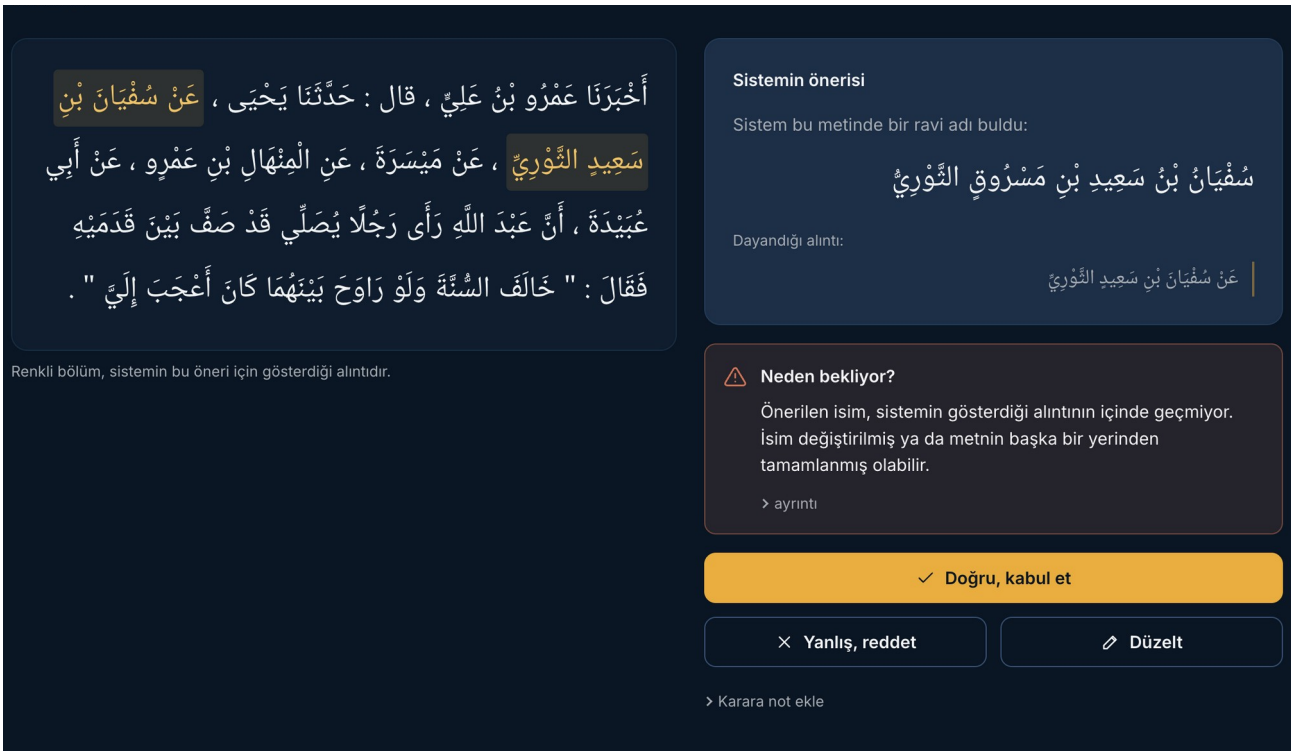


Figure 2. Detail of the existing prototype review screen, showing the source, proposal, failed check and decision actions. This repository screenshot demonstrates the interface, not completed research participant review.

Upload accepts raw text and an external reference; extraction selects the rule-based or configured model method. A run with unresolved items shows the review step before graph writing. The reviewer inspects the quoted source, considers reason codes, corrects a value if needed and records a decision. Model confidence remains supporting information rather than a probability of truth.

Screen flow and expected behaviour

Screen	Main action	Visible result
Project and upload	Create an isnad project; load a source and reference.	Source count, import errors and duplicate references.
Extraction	Select parser or available model; start a run.	Run state, candidate count and unresolved review items.
Review	Read the quotation and reason; accept, reject or edit.	Recorded action and updated queue; graph write remains blocked while unresolved items remain.
Read chains	Open a committed hadith; inspect a narrator or link.	Chain order, source evidence and human or policy acceptance basis.

Before acceptance, the group will check Arabic right-to-left display, mixed-script identifiers, keyboard access, empty/error/loading states and long quotations. The review screen also needs reliable duration capture and a clear distinction between human approval and automatic policy acceptance. A user must be able to return from a chain link to its source without editing the source text.

7. Development Environment and Repository

Repository URL	https://github.com/GiraffeGraph/giraffegraph
Branching / commit convention	One public group repository; development on main, with no long-lived branches. Members commit under their own accounts. Existing commit subjects use descriptive prefixes such as docs(project):.
Build and run instructions	Install Git, Docker and Compose v2. Clone the repository; enter giraffegraph-core; copy .env.example to .env; configure any optional model key; run docker compose up --build.
Tools and CI	Python 3.12 and uv; Node.js 22 and npm for local frontend builds; pytest, ruff, mypy; GitHub Actions. An IDE is a member choice. No mandatory issue-tracking process has yet been fixed.

Compose starts postgres, neo4j, backend, worker and dashboard. The application is available at localhost:3000; the API and generated API documentation are at localhost:8000 and /docs. Database migrations run at backend startup. Without model credentials the rule_based_isnad provider supports the full extraction workflow; the default noop provider deliberately returns no candidates [3].

For backend verification, run uv sync --extra dev, then uv run pytest -q, uv run ruff check src tests and uv run mypy src inside backend. For the frontend, run npm ci and npm run build inside frontend. The CI definition contains backend checks and a frontend build; the latest remote CI status was not re-run for this report.

8. Work Completed So Far

The following status is against the proposal work packages, not against the six-month research grant. Existing code is preliminary infrastructure; it is not proof that group review or participant evaluation is complete. Completion percentages are not estimated because the work packages do not have a fixed effort denominator [1], [3].

WP	Work package	Responsible	Status	% done
WP1	Architecture, data model, UI sketches and technology decisions	Yahya Efe; all review	Design documented; prototype screens exist; supervisor review pending.	Not quantified
WP2	Guideline, double annotation of 50 hadiths, expert decisions and agreement	Deniz; Alperen; experts	Kit prepared. Annotation and expert assessment have not started.	Not quantified
WP3	Midterm report and supervisor discussion	All	Report prepared; discussion at Supervisor Meeting 4 pending.	Not quantified
WP4	Group review and testing of segmenter, checks and Prophet rule	Alperen; Yahya Efe	Core implemented and unit-tested; group acceptance and Prophet rule pending.	Not quantified
WP5	Use review screen, test decision record and chain view	Deniz; Alperen; Yahya Efe	Interface exists. Measured reviewer session has not started.	Not quantified
WP6	100-hadith integration, S1–S5 checks, report, README and demo	All; Yahya Efe scoring	Scorer and README exist. Formal acceptance run and final demo not started.	Not quantified

Backend verification on 9 November 2026 completed with 273 passed and 12 skipped in 3.98 seconds. The skipped tests require PostgreSQL and Neo4j credentials and were not executed in this session. This verifies the current offline test suite; it does not certify the persistent end-to-end flow or the research hypotheses.

8.1 Preliminary measurements and their limits

The 12 October experiment compared four configurations on the same 300 Arabic hadiths, 50 from each of six books, against 1,778 Ifta reference narrator mentions. The following figures reproduce the saved comparison table [4]. M1 is normalized exact matching; M2 permits equality, substring containment or token F1 ≥ 0.60. Intervals use hadith-level bootstrap resampling.

Configuration	M2 precision / recall	M2 F1 and 95% interval
Rule-based parser	0.968 / 0.881	0.923 [0.910, 0.935]
Schema-only LLM, whole hadith	0.914 / 0.868	0.890 [0.865, 0.914]
Isnad-aware LLM, whole hadith	0.997 / 0.919	0.956 [0.946, 0.966]
Isnad-aware LLM, per chain	0.998 / 0.903	0.948 [0.936, 0.959]

These are agreement scores with platform labels, not human-verified accuracy. They describe candidate output before human review and are not the planned stage-by-stage ablation. The recorded model identifier is claude-haiku-5-5 with low effort. No human reviewed those runs. In the schema-only run, the value-in-quotation check flagged 155 of 1,688 narrator proposals, including 122 of 145 label disagreements; 33 flagged proposals matched the labels. Many differences were name transformations rather than invented narrators. The check cannot establish identity or religious authenticity [3], [4].

9. Plan for the Rest of the Semester

The main dependency is a usable human reference before scoring the reviewed system. Weeks below are course weeks; assignments recover the delayed annotation work without replacing the proposal targets. The schedule will be reviewed with the supervisor at Meeting 4.

Weeks	Planned work and exit evidence	Responsible
9–10	Review the guideline with domain experts; calibrate on a small sample, then independently label the same 50 hadiths without system output. Stabilize decision targeting before large review sessions. Review segmenter, checks, manual-entry bypass and Prophet rule. Exit: paired labels, initial disagreement list and reviewed fixes.	Deniz and Alperen: annotation; Yahya Efe: records/checks; all: expert coordination.
11–12	Obtain independent expert adjudication; report contested items and annotator agreement. Run one blinded reviewer session using a person who did not create that hadith's reference. Test edits, durations and chain evidence. Exit: frozen reference, decision log and review usability issues.	Deniz: review UI; Alperen: chain view; Yahya Efe: scorer/records; experts: adjudication.
13	Run parser, LLM, LLM with checks and LLM with checks plus human review on the reference. Audit every committed fact for evidence and acceptance. Exercise real-database retries and the 100-hadith workflow. Exit: run manifests, checks against S1–S4 and defect list.	All; Yahya Efe: metrics; Alperen: integration; Deniz: review session.
14–15	Repeat failed acceptance checks after fixes; perform clean-machine setup; reproduce supporting-study figures for S5; finish final report, README and demo. Exit: archived acceptance bundle, reproducible results and working preliminary prototype.	All, each committing their own work.

If expert availability slips, retain the 50-hadith target and escalate scheduling immediately; freeze the labels that are ready and mark remaining items unresolved. Unadjudicated or contested items will not silently enter the main gold-standard score. The larger grant experiments continue after the preliminary prototype, with their original hypotheses and targets preserved [2].

9.1 Evaluation and demonstration acceptance

The proposal's S1–S5 criteria remain the completion checklist [1]. S1 requires a logged 100-hadith upload-to-canonical-graph run with human actions confined to the review screen. S2 requires evidence and an acceptance basis for every canonical fact. S3 requires strict/loose precision, recall and F1 for four conditions on the 50-hadith reference, with hadith-level confidence intervals and per-check error-catching figures compared with confidence at the same review share.

S4 reports annotator mutual F1, expert agreement, contested items and labelling time. Two group members label independently; three experts are intended to adjudicate disagreements independently, with majority decisions retained and unresolved cases reported separately. A sample of annotator agreements is also checked. S5 requires reproducing the supporting studies from scripts and reporting their limitations. None of these final acceptance criteria is declared complete at midterm.

The final demonstration will create an isnad project, import a fixed 100-hadith batch, select a provider and run extraction. It will show a candidate whose quotation or name check fails, record accept/reject/edit decisions with duration, and write the resolved run to the graph. The group will open a chain, trace a transmission link to its source and acceptance basis, repeat a commit to check idempotency, and export the run for scoring. Unresolved runs must stay out of the canonical graph.

10. Risks

Risk	Likelihood	Impact	Mitigation
Annotation and expert work already delayed	High	High	Start recovery in Week 9; coordinate expert slots immediately; measure initial annotation time; review progress weekly. Owner: all.
Platform labels mistaken for a human reference	High	High	Use existing scores only as preliminary agreement; freeze independent labels before evaluation; report contested items. Owners: Deniz and Alperen.
Review effort lost when extraction is repeated	High	High	Current decisions target run-specific candidate IDs. Stabilize content/schema/source targeting, preserve history and test reuse boundaries before scaling review. Owner: Yahya Efe.
Name-based nodes merge different narrators	High	High	Expose the limitation in chain views; keep surface-name evaluation separate from person identity; do not claim authority linking. Owner: Alperen.
Missing evidence or permissive manual entry reaches graph	Medium	High	Apply source checks to all measured write paths; refuse missing provenance; audit S2 and reviewer overrides. Owner: Yahya Efe.
PostgreSQL/Neo4j state diverges after a failure	Medium	High	Inject commit failures; test deterministic retries; reconcile run status and graph delta before accepting the batch. Owners: Yahya Efe and Alperen.
Provider output format, availability or cost changes	Medium	Medium	Keep parser fallback, spending limit and cached responses; validate adapters before experiments; save prompt/model identifiers. Owner: Yahya Efe.
Text redistribution rights remain unclear	Medium	High	Track source/edition/licence per dataset; distribute only permitted material and derived identifiers/labels; select software licence before public release. Owner: all.

The risk that no repository or prototype skeleton exists is closed: both are present. Scientific validity and human review remain open risks. The current application trusts typed reviewer names and has no user authentication, so demonstrations remain in a controlled group environment; public multi-user use requires a separate access-control design.

11. References

[1] GiraffeGraph Group 10, “CSE 491 Project Proposal,” 18 Oct. 2026. Project record: wiki/thesis/proposal/proposal.md.

[2] Y. E. Kuruçay, D. Büyükşahin and A. Ulukaya, “Büyük Dil Modelleriyle Bilgi Grafı Üretiminde Doğrulama Denetimlerinin ve İnsan İncelemesinin Etkisi: Hadis İsnad Zincirleri Üzerine Bir Uygulama,” TÜBİTAK 2209-A research proposal, 2026. Project record: wiki/thesis/tubitak/form.md.

[3] GiraffeGraph Group 10, “GiraffeGraph Core,” source code, README, architecture documents and frontend screen archive. Repository: https://github.com/GiraffeGraph/giraffegraph. Inspected at commit 23d249b, 9 Nov. 2026; local backend pytest verification on the same date.

[4] GiraffeGraph Group 10, “Run comparison,” 12 Oct. 2026. Experiment record: giraffegraph-core/evaluation/results/2026-10-12-sixbooks300/compare-ifta.md; analysis and limits: wiki/evaluation/engine-experiments.md.

[5] M. Mghari, O. Bouras and A. El Hibaoui, “Sanadset 650K: Data on Hadith narrators,” Data in Brief, vol. 44, 108540, 2022. doi: 10.1016/j.dib.2022.108540.

[6] A. B. Kamran, B. Abro and A. Basharat, “SemanticHadith: An ontology-driven knowledge graph for the hadith corpus,” Journal of Web Semantics, vol. 78, 100797, 2023. doi: 10.1016/j.websem.2023.100797.

[7] FastAPI, “Features.” https://fastapi.tiangolo.com/features/. Accessed 9 Nov. 2026.

[8] PostgreSQL Global Development Group, “PostgreSQL 16 Documentation: Transaction Isolation.” https://www.postgresql.org/docs/16/transaction-iso.html. Accessed 9 Nov. 2026.

[9] Neo4j, “What is a graph database.” https://neo4j.com/docs/getting-started/graph-database/. Accessed 9 Nov. 2026.

[10] Vercel, “Next.js 15 Documentation.” https://nextjs.org/docs/15. Accessed 9 Nov. 2026.

[11] S. Tsaneva, D. Dessì, F. Osborne and M. Sabou, “Knowledge graph validation by integrating LLMs and human-in-the-loop,” Information Processing & Management, vol. 62, no. 5, 104145, 2025. doi: 10.1016/j.ipm.2025.104145.

Use of AI assistants

AI assistants were used in preliminary implementation, literature organization, evaluation scripts and report drafting. The group is responsible for checking the code, citations, numerical claims and submitted text. AI-generated source descriptions and draft results do not replace independent annotation or expert judgement.

Supervisor Assessment

Supervisor name	
Assessment	Satisfactory ☐ Satisfactory with revisions ☐ Not satisfactory ☐
Comments	
Date / signature	